Elaborazione 1

Introduzione

Durante la prima iterazione della fase di elaborazione si é ritenuto opportuno concentrarsi sulle funzionalitá chiave del sistema. In particolare si è deciso di iniziare con analisi e progettazione dei seguenti casi d'uso:

- UC1: Associa Dispositivo
- UC6: Comanda Dispostivi (Vengono per il momento tralasciate le sequenze di comandi)

Questa scelta ci permette inoltre di scongiurare i rischi dovuti alla comunicazione tra il sistema ed i vari dispositivi. Tali rischi derivano principalmente dalla eterogeneitá funzionale ma anche comunicativa dei dispositivi che, fanno cose diverse e potenzialmente comunicano in maniera diversa e attraverso tecnologie e protocolli diversi. Anche qualora si riuscisse a rappresentare una tale diversitá, ció presumibilmente renderebbe il sistema ingestibile, fortemente accoppiato e quindi difficilmente aggiornabile.

Si é optato invece per un approccio meno specifico alla rappresentazione ed al controllo dei dispositivi.
Si generalizza il concetto di dispositivo evitando di specificarne le caratteristiche in fase di progettazione.
Come capire allora le funzionalità o i comandi compatibili?
Nella fase di associazione il dispositivo, tra le altre informazioni, trasmette anche una descrizione del proprio stato che specifica l'insieme dei parametri di stato e di tutti i vincoli che ad essi si applicano. Il sistema conserva la rappresentazione dei dispositivi associati e la descrizione di ciascuno di essi. Ciò semplifica anche l'individuazione dell'insieme dei comandi che il sistema può impartire in quanto vengono definiti dallo stato stesso di ciascun dispositivo. Per esempio se lo stato di un dispositivo e la sua descrizione sono rispettivamente:

```
//stato
{
    "power": "on",
    "luminosita": 100
}
//descStato
{
    "power":
        {
            "nome":"power",
            "tipo":"bool",
            "unitaMisura":null,
            "readOnly":false,
            "min":null,
            "max":null,
            "valoriAccetabili":["on","off", true, false]
        },
    "luminosita":
        {
            "nome":"luminosita",
            "tipo":"int",
            "unitaMisura":"%",
            "readOnly":false,
            "min":0,
            "max":100,
            "valoriAccetabili":null
        },
}
```

Il comando sarà semplicemente:

```
//comando
{
    "param": "power",
    "valore": "on"
}
//oppure
{
    "param": "luminosita",
    "valore": 77
}
```

Ovviamente, ogni dispositivo avrà parametri di stato in generale diversi per cui sarà necessario che la UI del sistema indirizzi l'utente verso l'utilizzo dei comandi appropriati.

UC1

Analisi

Relativamente al caso d'uso scelto (UC1), dopo una valutazione preliminare è stato possibile identificare le seguenti classi concettuali da cui è stato ricavato il Modello di Dominio in figura.

- **SmartHub**: rappresenta il sistema SmartHub
- **Casa**: rappresenta il dominio del sistema all'interno del quale si trovano le stanze
- **Stanza**: rappresenta la posizione di un dispositivo all'interno della casa
- **RegistroDispositivi**: lista dei dispositivi associati
- **Dispositivo**: dispositivo fisico o simulato
- **DescDispositivo**: ogni dispositivo ne ha una: specifica le informazioni rilevanti
- **ParamStato**: parametro di stato; descrive lo stato funzionale del dispositivo; ogni dispositivo ne può avere più di uno.
- **DescParam**: descrizione di un parametro di stato: ogni parametro deve essere descritto

Modello di Dominio

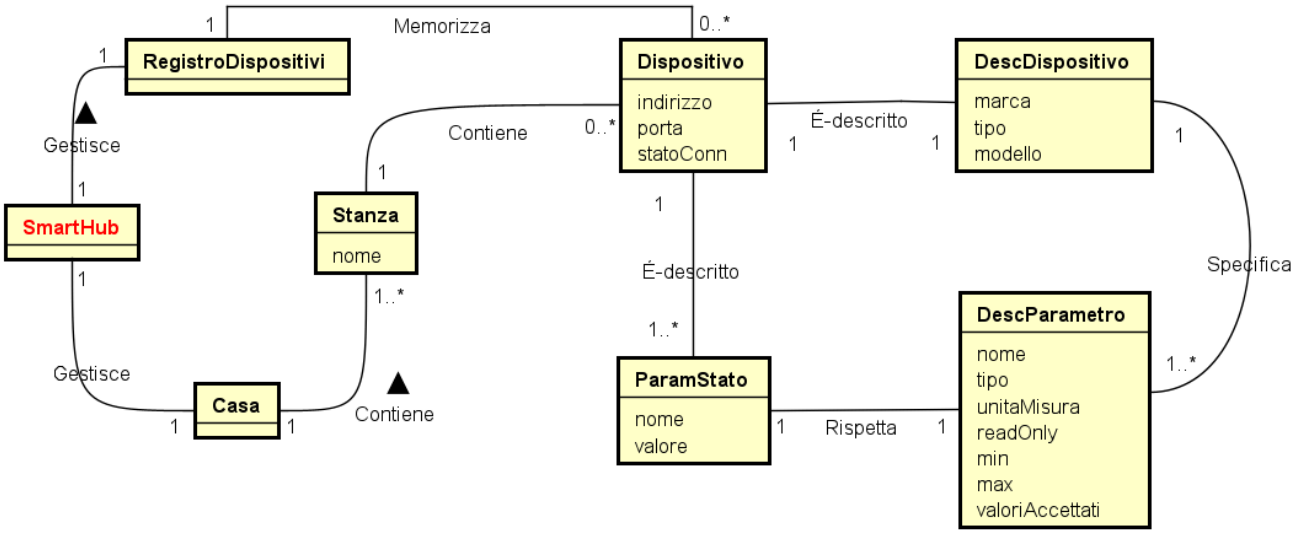
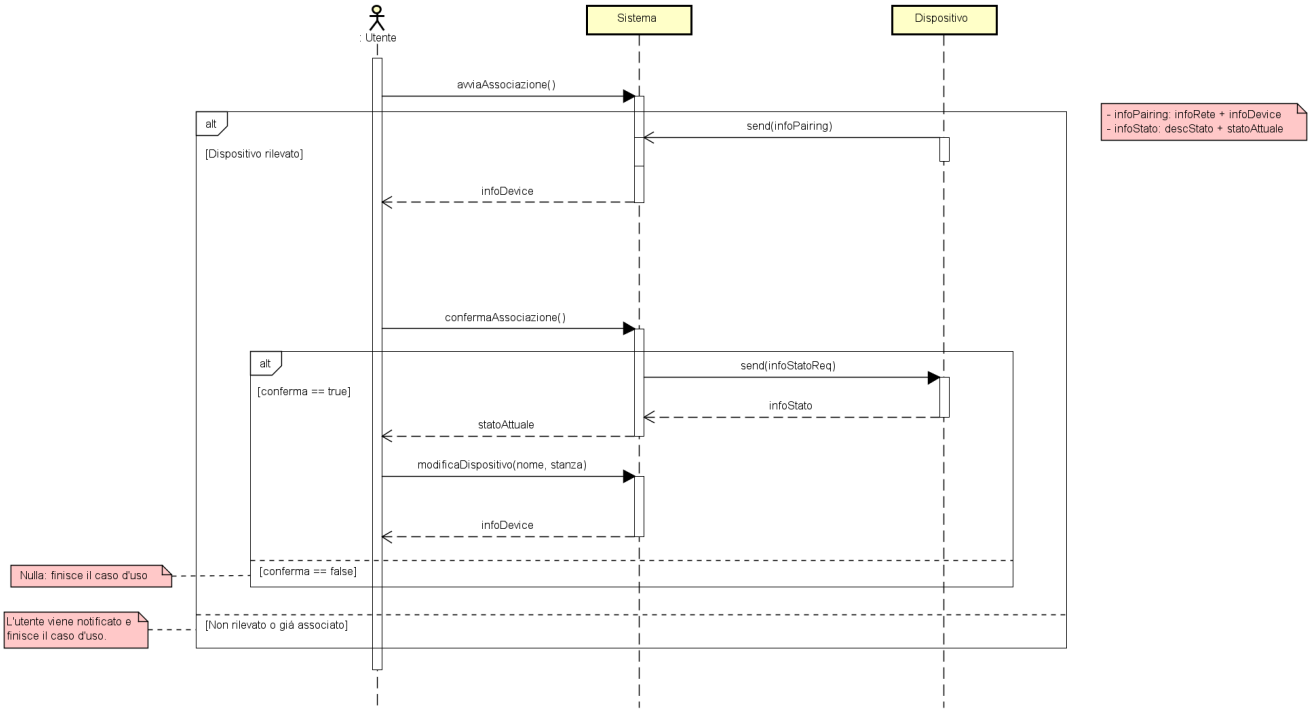


Diagramma di Sequenza di Sistema (SSD)

Si rappresentano nel seguente diagramma le interazioni tra il sistema ed eventuali attori o sistemi esterni nello svolgersi dello scenario principale di successo del caso d'uso.



Contratti delle operazioni

- **Contratto CO1**: avviaAssociazione
  - **Operazione**: avviaAssociazione()
  - **Riferimenti**: UC1: Associa Dispositivo
  - **Pre-condizioni**: nessuna
  - **Post-condizioni**: L'hub è in ascolto su una certa porta.
- **Contratto CO2**: confermaAssociazione
  - **Operazione**: confermaAssociazione(infoDevice)
  - **Riferimenti**: UC1: Associa Dispositivo
  - **Pre-condizioni**: Un dispositivo è stato rilevato
  - **Post-condizioni**:
    - Un'istanza d di Dispositivo è stata creata.
    - Un'istanza desc di DescDispositivo è stata creata.
    - Istanze di ParamStato sono state create e associate a d.
    - Istanze di DescParam sono state create e associate a desc.
    - Il Dispositivo d è stato associato alla DescDispositivo desc.
    - Il Dispositivo d è stato aggiunto al RegistroDispositivi.
- **Contratto CO3**: modificaDispositivo
  - **Operazione**: modificaDispositivo(nome, stanza)
  - **Riferimenti**: UC1: Associa Dispositivo, UC2: Gestisci Dispositivo
  - **Pre-condizioni**: Istanza di Dispositivo creata ed in attesa di configurazione.
  - **Post-condizioni**:
    - L'attributo nome di d è adesso nome.
    - L'istanza di d è adesso associata all'istanza di Stanza specificata da stanza.

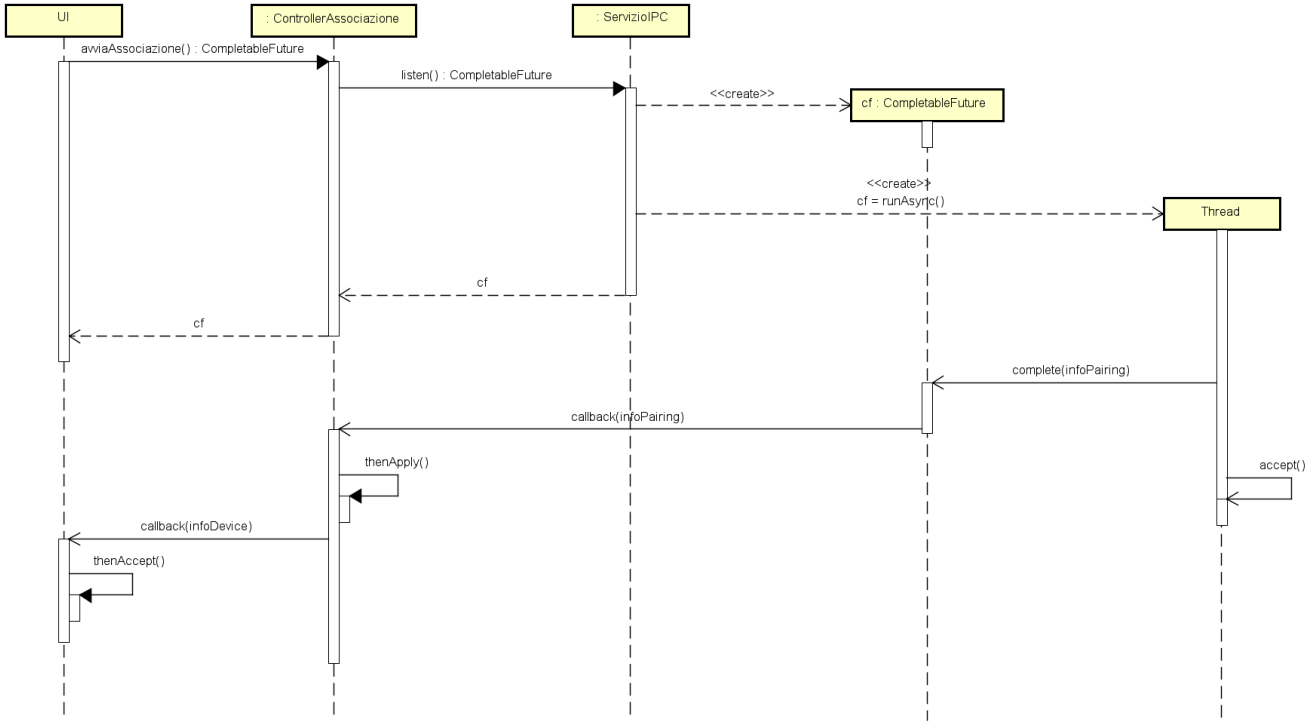
Progettazione

Diagrammi di sequenza

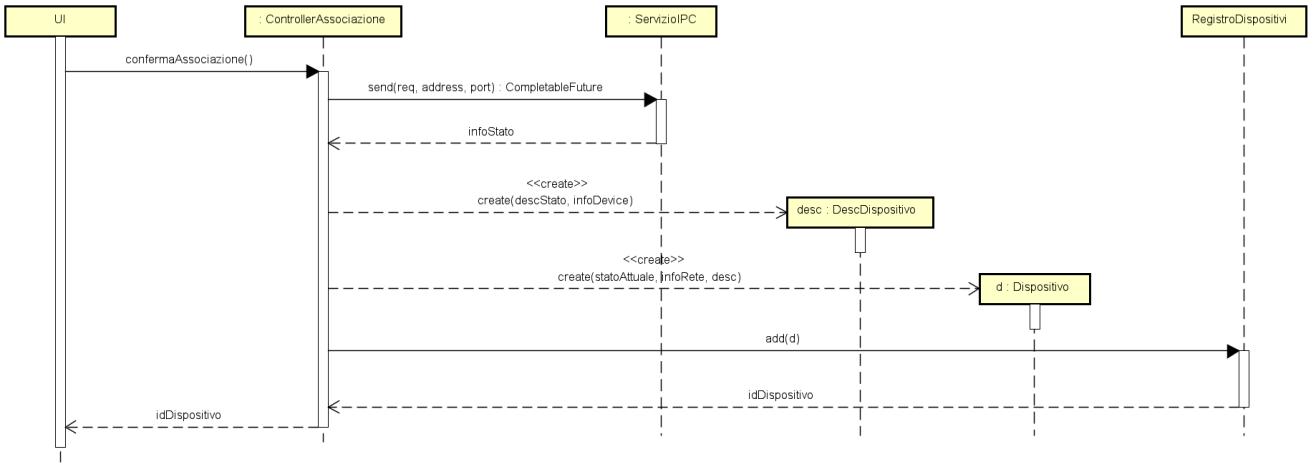
Op1: avviaAssociazione()

Nel diagramma seguente si è voluto esplicitare il funzionamento dei `CompletableFuture`, la classe introdotta a partire da Java 8 a supporto della programmazione asincrona. All'atto pratico, l'invocazione del metodo

restituisce immediatamente l'oggetto `CompletableFuture`, non bloccando il flusso di esecuzione principale. L'elaborazione vera e propria viene demandata a un thread separato in background, il quale si occuperà di valorizzare e completare tale oggetto in un secondo momento. Nei diagrammi di sequenza successivi, per alleggerire la notazione, le dinamiche di questo meccanismo verranno omesse. Le chiamate verranno rappresentate in forma standard, indicando esplicitamente il tipo di ritorno asincrono (`CompletableFuture<T>`) a conferma della natura non bloccante dell'operazione.



Op2: confermaAssociazione()



Op3: modificaDispositivo()

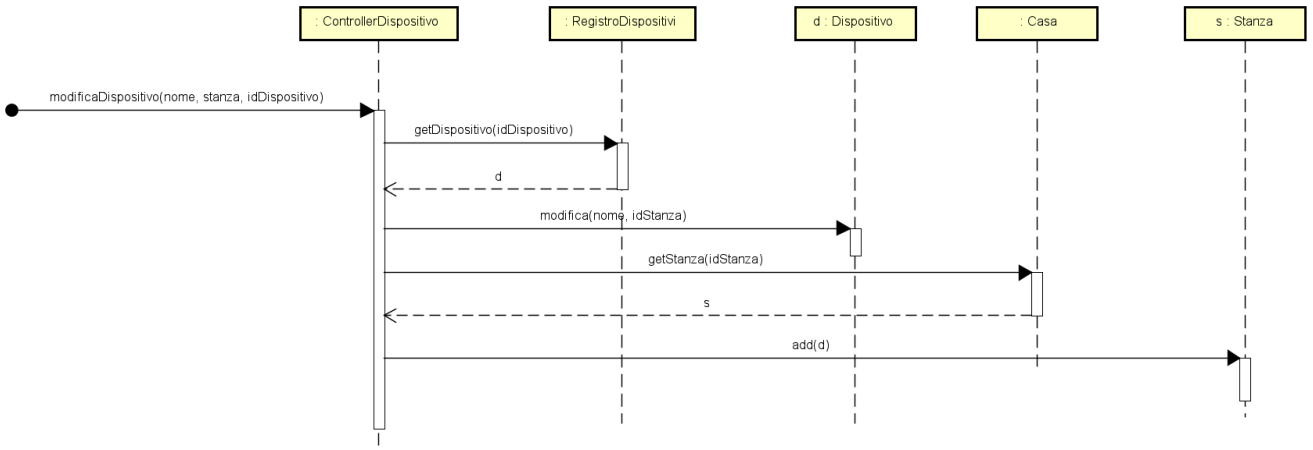
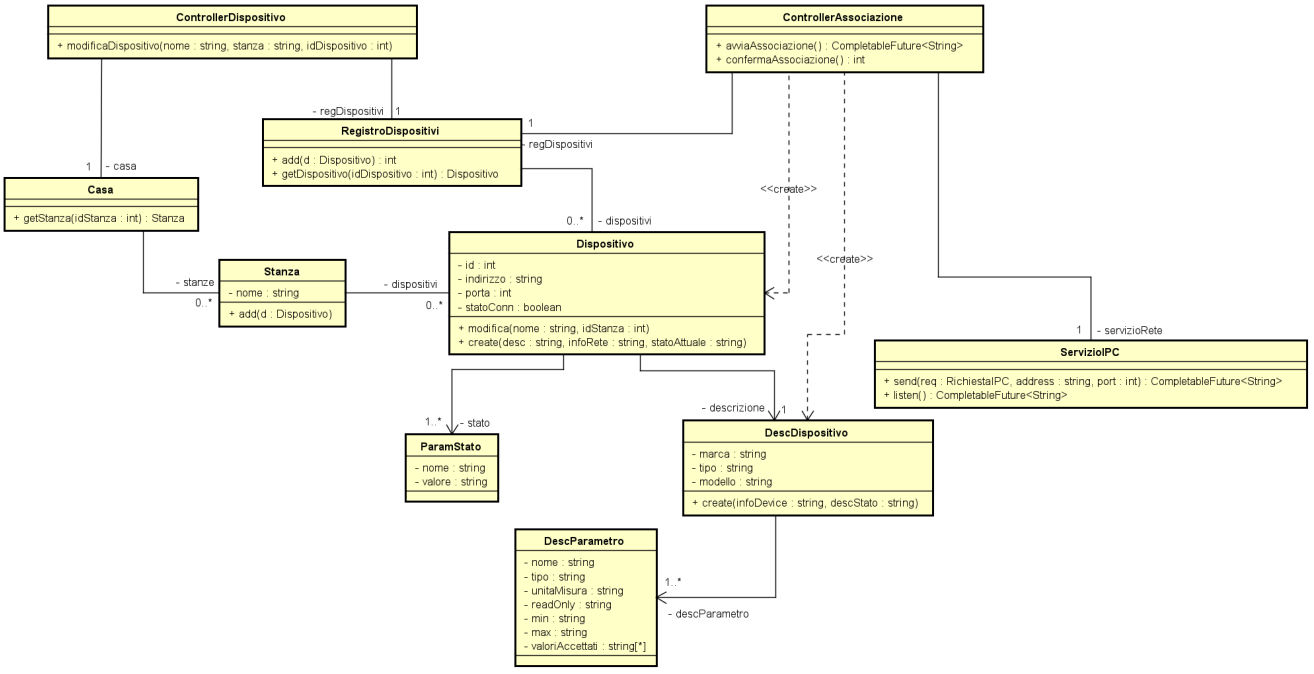


Diagramma delle classi



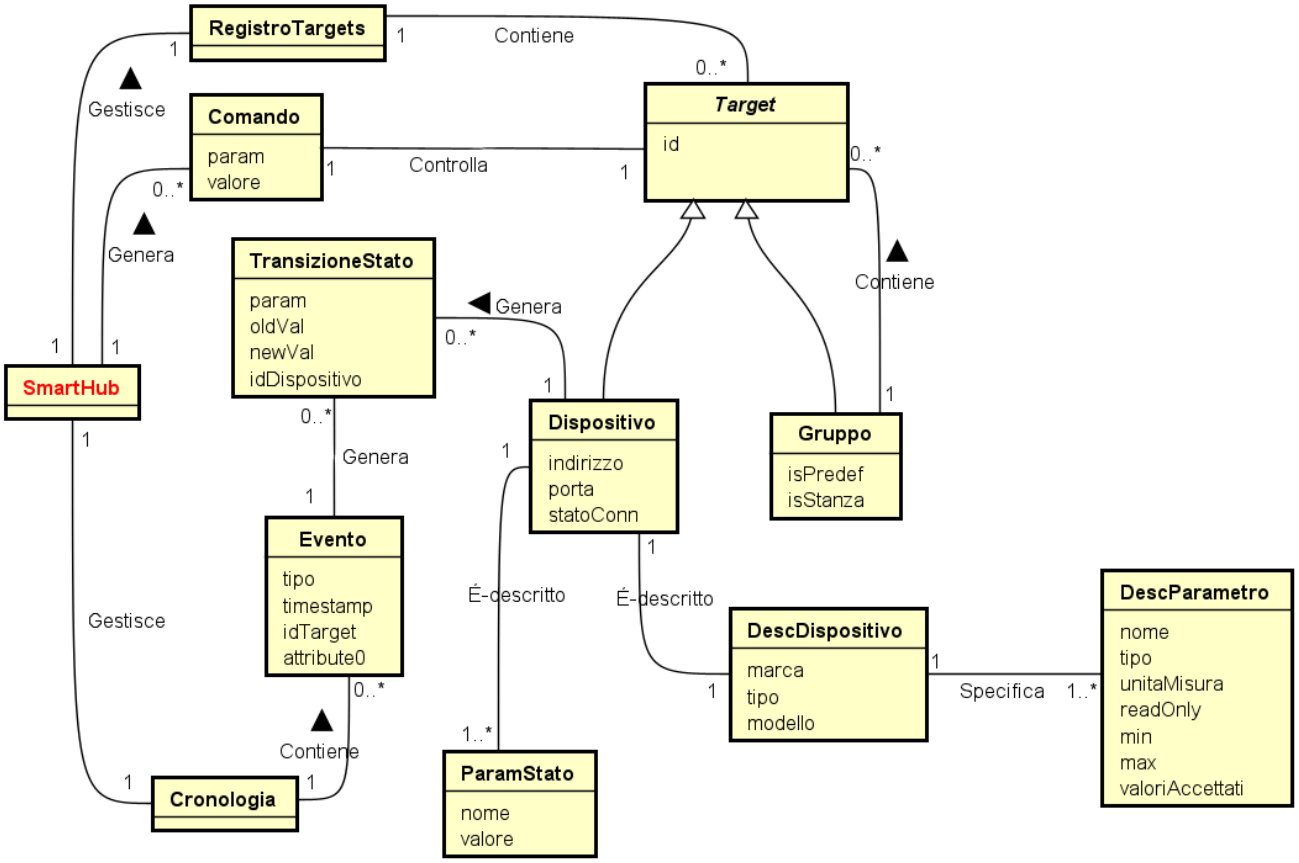
UC6

Analisi

Come si vede in figura il dominio di UC6 ha inglobato ed esteso il modello di dominio del caso d'uso analizzato precedentemente. In particolare sono stati introdotti i seguenti concetti:

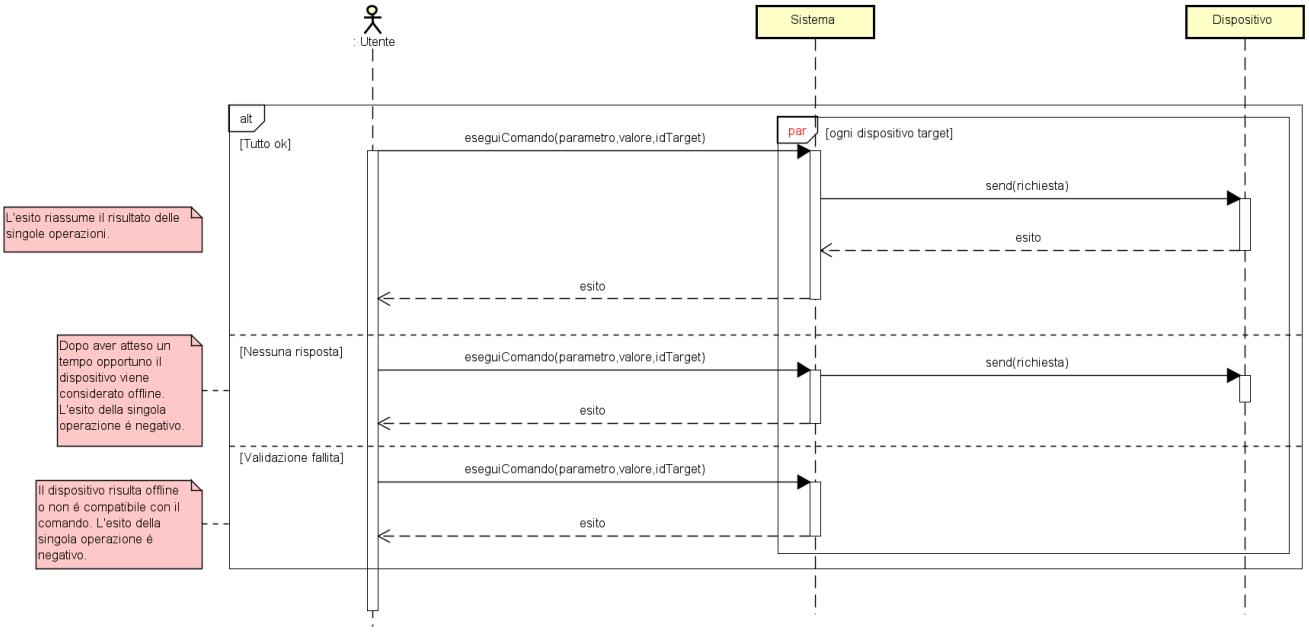
- **Gruppo**: insieme di dispositivi definito dall'utente; ha inglobato il concetto di Stanza
- **Target**: Dispositivo singolo o Gruppo di dispositivi
- **RegistroTargets**: lista di Target registrati nel sistema; ha inglobato i concetti di RegistroDispositivi e Casa
- **Comando**: istruzione per il controllo di dispositivi compatibili
- **TransizioneStato**: variazione di un parametro di stato; può essere generata da un comando
- **Evento**: episodio di interesse per il sistema; una TransizioneStato genera sempre un evento
- **Cronologia**: storico degli eventi del sistema in ordine di occorrenza

Modello di Dominio



Diagrammi di Sequenza di Sistema (SSD)

Si rappresentano nel seguente diagramma le interazioni tra il sistema ed eventuali attori o sistemi esterni nello svolgersi dello scenario principale di successo del caso d'uso.



Contratti delle operazioni

- **Contratto CO1**: eseguiComando
  - **Operazione**: eseguiComando(param, valore, idTarget)
  - **Riferimenti**: UC6: Comanda Dispositivi
  - **Pre-condizioni**: esiste un'istanza t di Target corrispondente a idTarget
  - **Post-condizioni**:

- Lo stato operativo delle istanze di Dispositivo coinvolte è stato aggiornato
- É stata creata un'istanza e di evento
- L'istanza e di Evento è stata aggiunta alla Cronologia.

Progettazione

Diagrammi di sequenza

Il calcolo della compatibilità dei comandi è demandato alle classe `Dispositivo` secondo il principio dell' Information Expert, in quanto unica detentrici delle informazioni sul proprio stato interno.

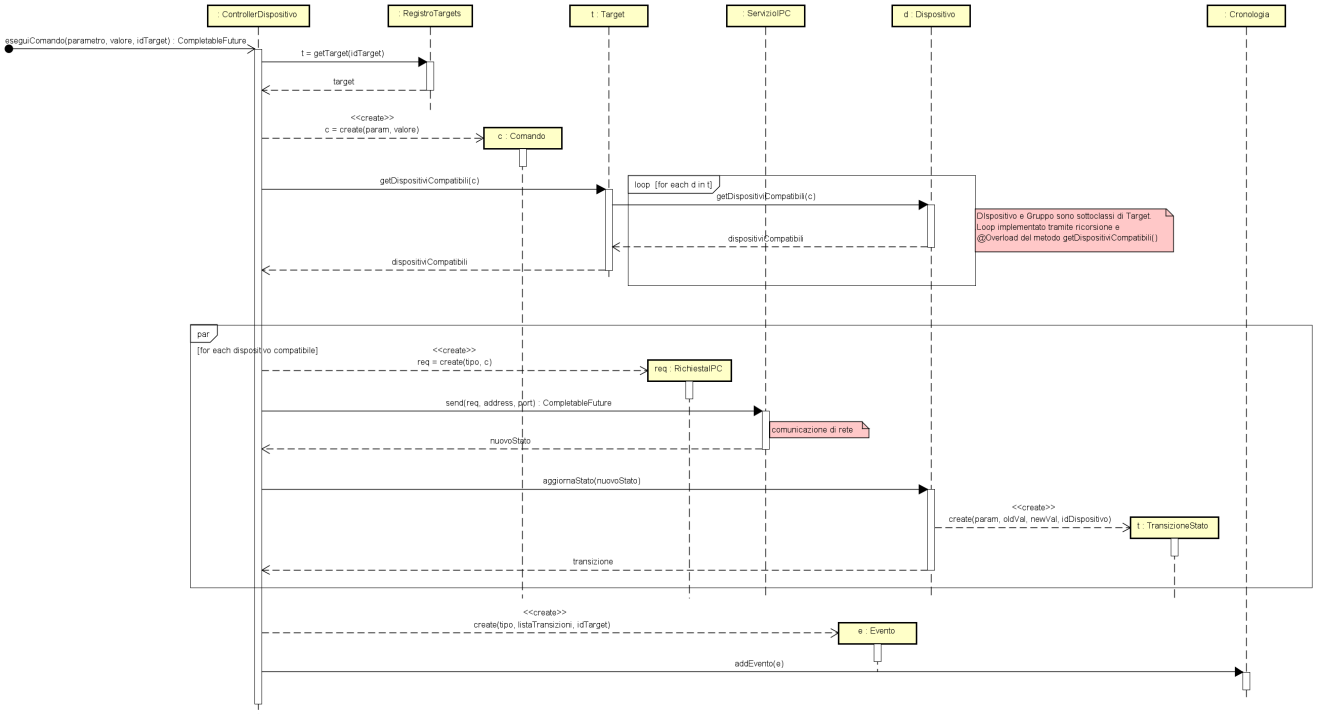
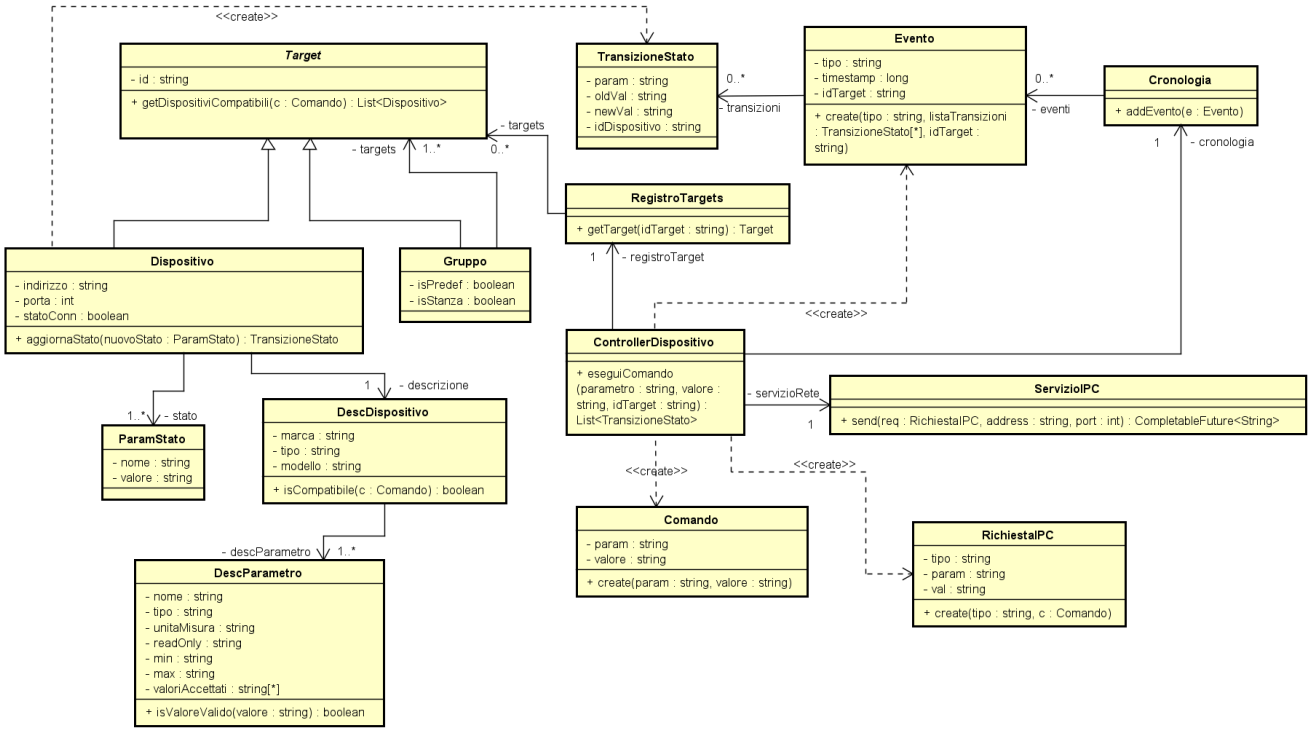


Diagramma delle classi



Implementazione

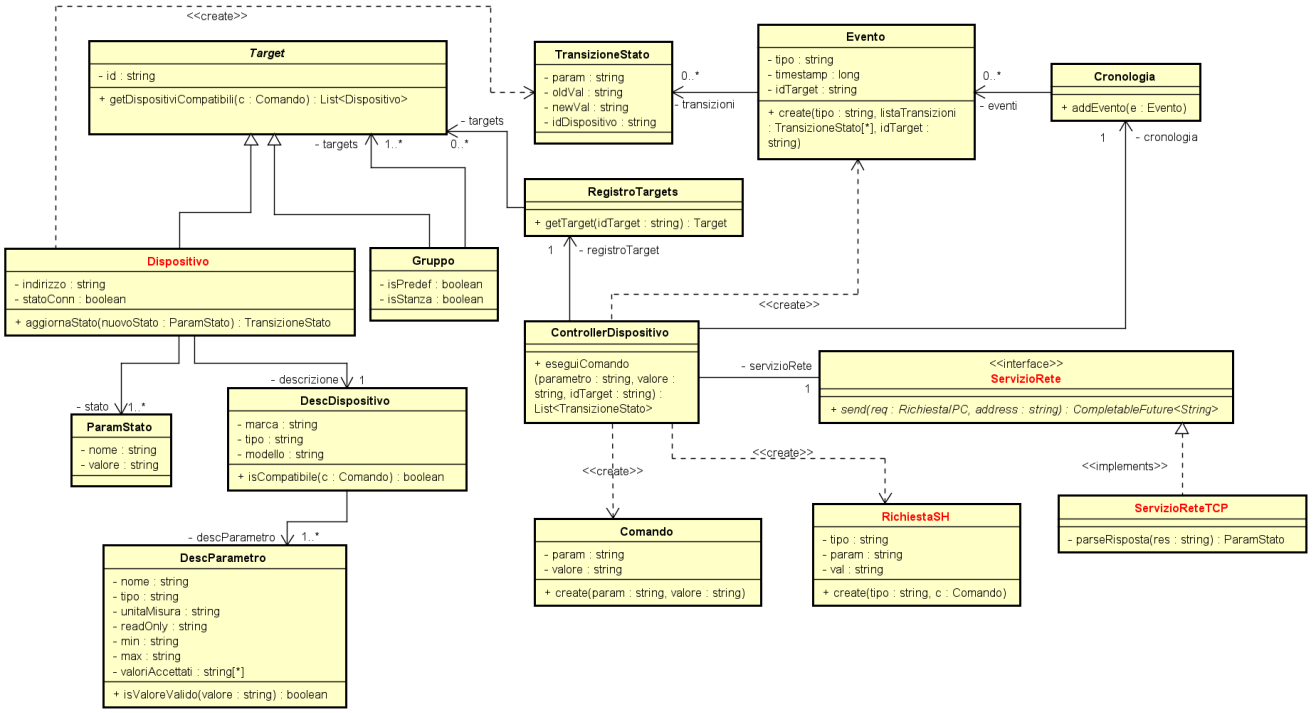
Durante la prima iterazione, lo sviluppo software si è concentrato sulla realizzazione del nucleo logico del sistema SmartHub, con particolare focus sulle funzionalità descritte nell'UC6 riguardanti l'invio dei comandi verso i dispositivi. É stata implementata una semplicissima UI testuale temporanea per l'interazione col sistema e, in un package a parte, delle classi che simulano alcuni dispositivi.

Refactor Elaborazione 1

Al fine di migliorare la coesione del codice, disaccoppiare i componenti e preparare il sistema a future estensioni, è stata eseguita una fase di refactoring strutturale. Gli interventi principali sono riassunti di seguito:

- **Extract Interface e Rename:** Per eliminare l'accoppiamento verso un'implementazione concreta, è stata astratta la logica di comunicazione applicando il pattern Extract Interface. È nata così l'interfaccia `ServizioRete`, mentre la classe originale `ServizioIPC` è stata rinominata in `ServizioReteTCP` per esplicitare la tecnologia utilizzata. Parallelamente, il payload di comunicazione è stato rinominato da `RichiestaIPC` a `RichiestaSH` per aderire meglio al linguaggio del dominio (SmartHub).
- **Generalizzazione dell'Indirizzamento:** All'interno della classe `Dispositivo`, gli attributi separati per IP e Porta sono stati unificati in un'unica stringa `indirizzo` (es. "ip:porta"). Questa modifica ha permesso di snellire la firma del metodo `send` e, soprattutto, di rendere il `ControllerDispositivo` del tutto agnostico: il controller passa un generico indirizzo testuale senza doversi preoccupare di come la specifica tecnologia di rete sottostante lo interpreterà.
- **Extract Method:** All'interno della classe `ServizioReteTCP`, la logica di invio dati era originariamente fusa con quella di interpretazione del risultato. Applicando il pattern Extract Method, la decodifica della risposta grezza è stata isolata nel nuovo metodo privato `parseRisposta(String res)`. Questo intervento rispetta il Single Responsibility Principle, rendendo il metodo di invio più leggibile e isolando la logica di estrazione del `ParamStato`.

Il seguente diagramma delle classi riassume le modifiche effettuate.



Note Progettuali

L'architettura del sistema SmartHub si sviluppa su più strati al fine di garantire un elevato disaccoppiamento e una rigida separazione delle responsabilità. Il sistema è suddiviso logicamente in quattro package principali:

- `ui`: Gestisce l'interazione con l'utente e raccoglie gli input, rimanendo totalmente agnostico rispetto alle regole di business.

- **`app`:** Agisce da strato applicativo e di orchestrazione. Coordina il flusso dei dati e funge da punto di ingresso del sistema, assemblando i vari componenti. Classi come `ControllerAssociazione` e `ControllerDispositivo` servono da intermediari per mantenere l'interfaccia utente completamente disaccoppiata dalla logica di dominio.
- **`domain`:** Contiene il cuore logico dell'applicazione. Al suo interno è stato implementato il Design Pattern *Composite* per la gestione dei destinatari: tramite l'astrazione `Target`, il sistema è in grado di trattare in modo del tutto trasparente e uniforme sia i `Dispositivi` singoli che i `Gruppi`, estraendo in autonomia solo i dispositivi fisici compatibili con l'azione richiesta.
- **`services`:** Si occupa della comunicazione fisica con l'esterno e delle utilities. L'effettiva trasmissione dei payload è stata delegata a questo strato attraverso il `ServizioIPC`, che gestisce l'inoltro tramite socket TCP isolando il dominio dai dettagli di rete.